Parcours des écoles d'ingénieurs Polytech (MAM3)

Introduction à la Programmation et à l'Algorithmique

Présentation : **H. Renard, D Lopez Pacheco**



Pour dynamiser et rendre interactif un cours en Cours/TD d'IPA ...

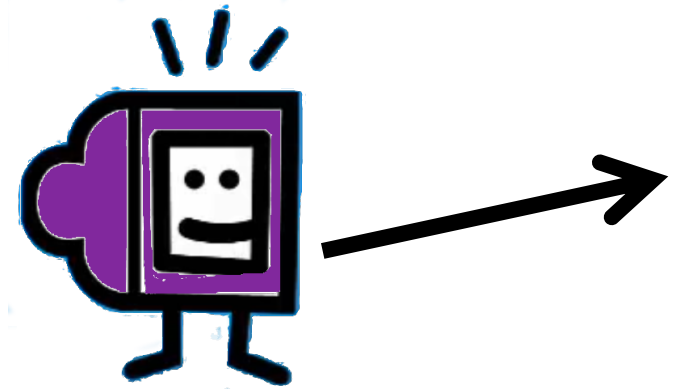
Les Objets

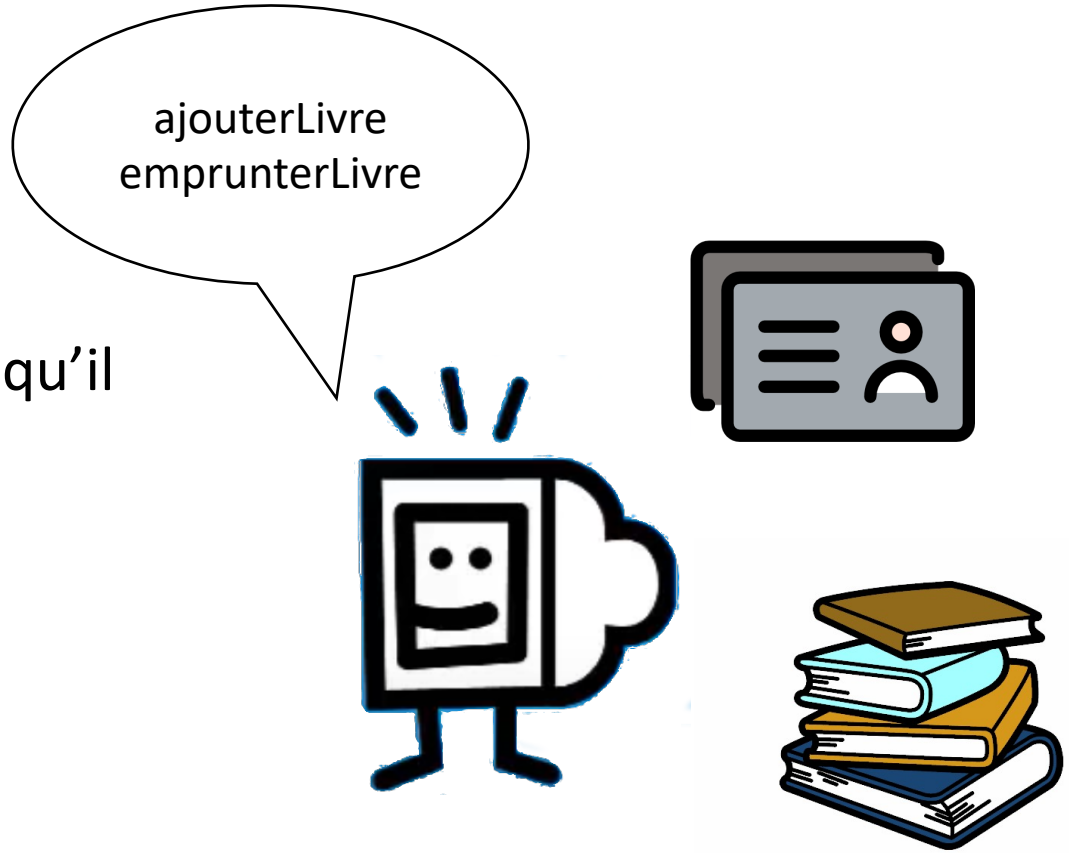
Objets/instances et classes, méthodes et attributs

* Caractéristiques d'un objet

✓ <u>Un objet se caractérise par</u> :

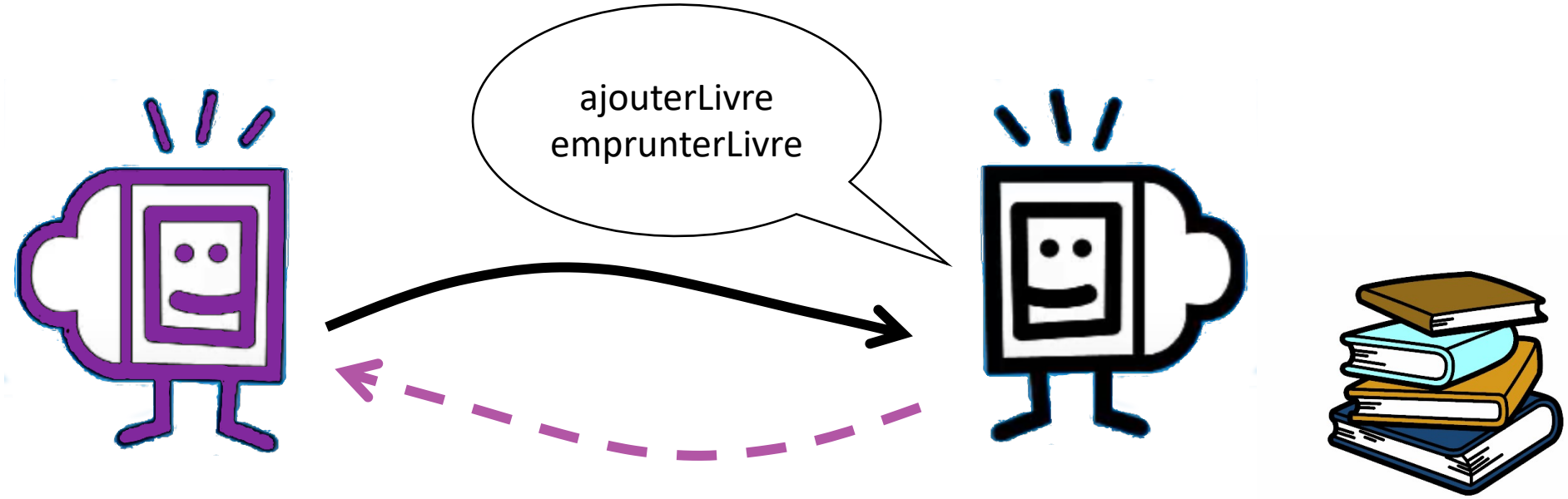
- ❑ son identité;
- ❑ ses données;
- ❑ les traitements qu'il sait réaliser et qu'il propose aux autres objets.





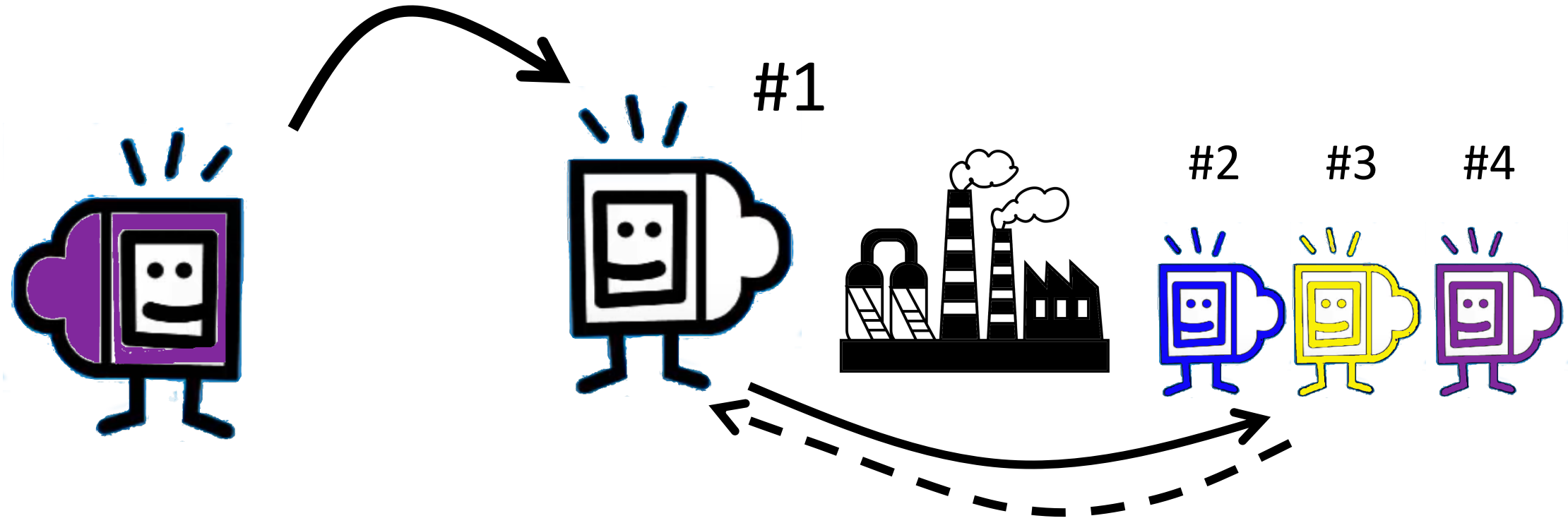
* Communication

- ✓ Les objets communiquent entre eux par échange de message;
- ✓ Le message le plus échangé est la demande de réalisation de traitement.



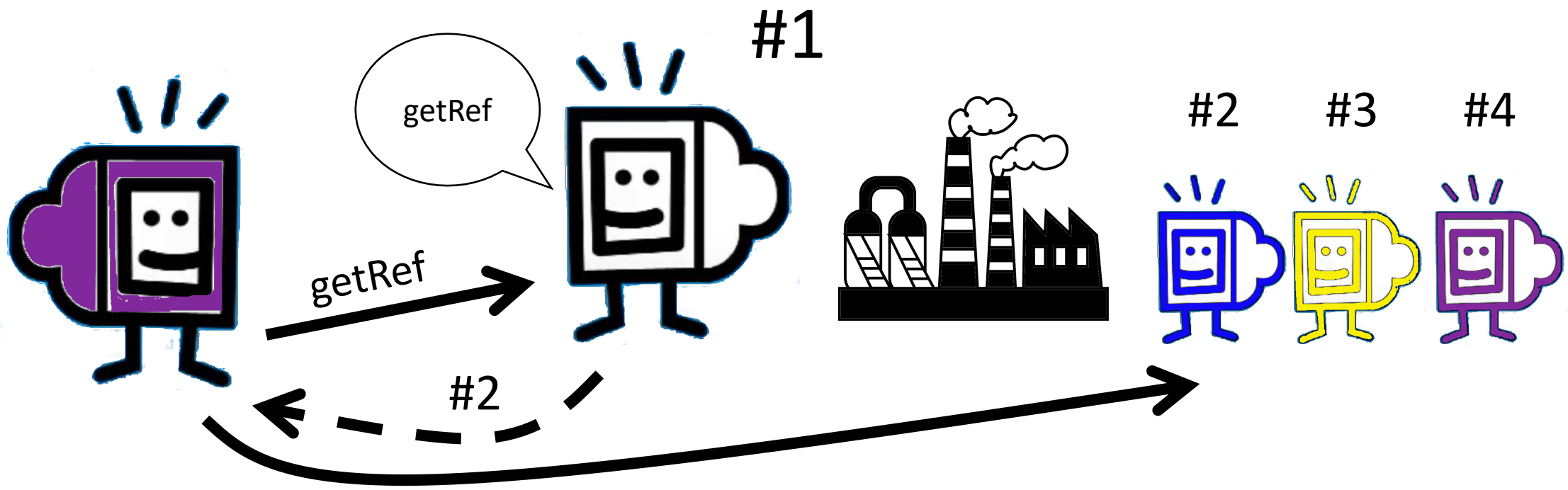
* Création d'objet et référence

- ✓ Chaque objet peut créer d'autres objets;
- ✓ Le créateur possède une référence vers les objets créés.



* Échange de références

- ✓ Les références sont des données;
- ✓ Il est possible de les transmettre via échange de message.



* Suppression d'objet

- ✓ Soit on peut supprimer les objets et alors il faut faire attention aux références qui pointent vers les objets supprimés (C++);
- ✓ Soit on ne peut pas supprimer les objets mais ceux-ci sont mis aux oubliettes si aucun autre objet ne les référence (Java).



Objets Java

Objets/instances et classes, méthodes et attributs

* Objets/instances et classes

✓Les classes représentent un ensemble d'objets similaires Ce sont les ***modèles des instances***

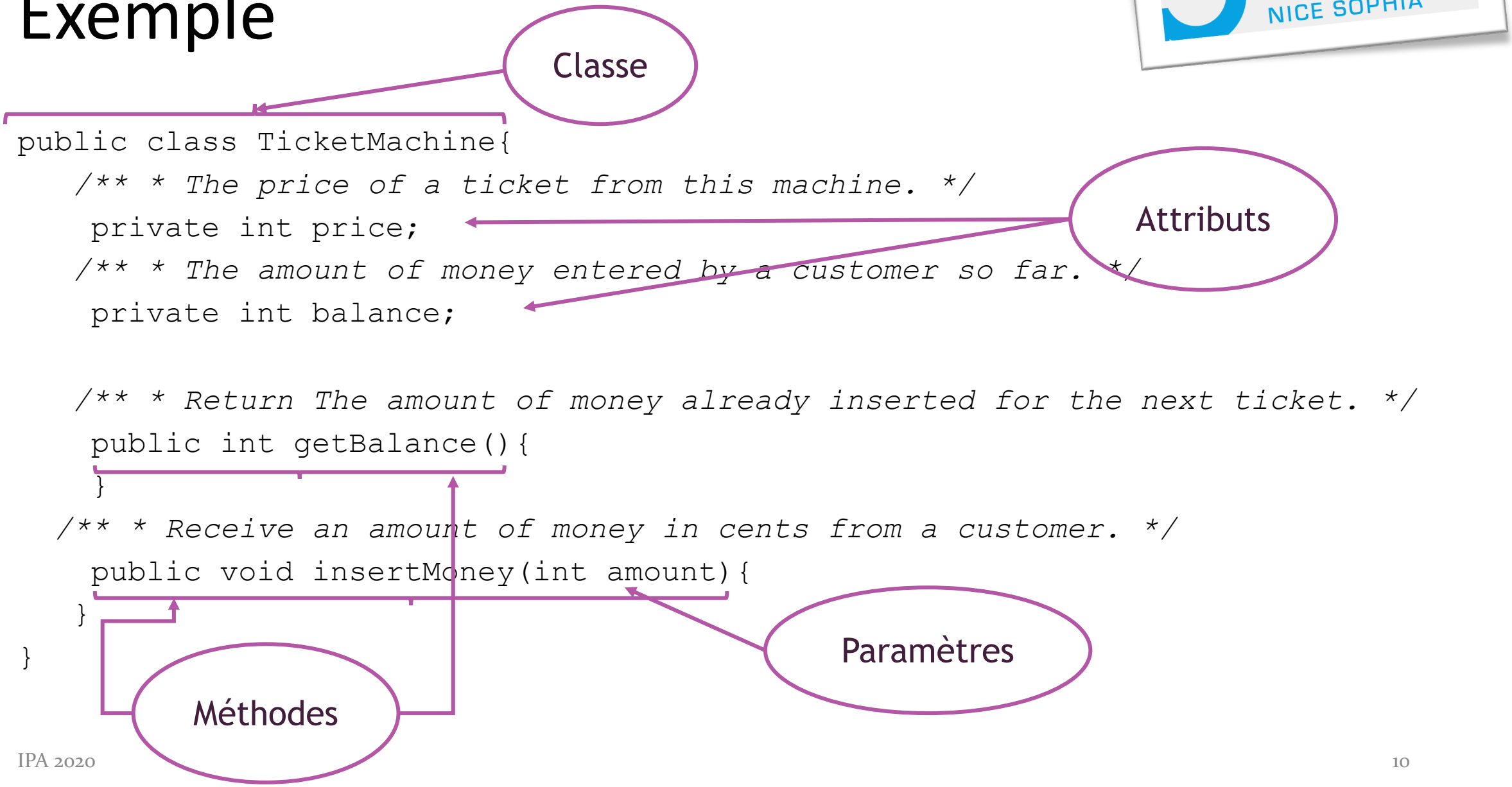
- ❑ Voiture;
- ❑ Carré;
- ❑ Échiquier;
- ❑ Case.



✓ Les objets (***instances***) représentent des entités du monde réel relatives au problème à traiter

- ❑ **Une** voiture rouge de marque Ferrari (course);
- ❑ **Un** carré de taille 4cm de côté (dessin);
- ❑ **Un** échiquier de 16 cases (jeu).
- ❑ Plusieurs instances peuvent être créées à partir d'une classe;

* Exemple




```
public class TicketMachine{
    /** * The price of a ticket from this machine. */
     private int price;
    /** * The amount of money entered by a customer so far. */
     private int balance;

    /** * Return The amount of money already inserted for the next ticket. */
     public int getBalance(){
     }
   /** * Receive an amount of money in cents from a customer. */
     public void insertMoney(int amount){
    }
}
```

* Constructeurs : initialisation des instances

Les ***constructeurs*** permettent de construire :

- ❑ les instances d'une classe, d'initialiser les variables d'instances;
- ❑ le nom du constructeur est le même nom que la classe;
- ❑ les valeurs des paramètres sont utilisées pour l'initialisation.

```
public TicketMachine(int ticketCost) {
        this.price = ticketCost;
        this.balance = 0;
        this.total = 0;
}
```

```
public TicketMachine(){
        this.price=7;
        this.balance=0;
        this.total=0;
        this.nomMachine="laStrada";
}
```

Instructions d'affectations



« this. » permet designer la variable d'instances

* Déclaration de constructeurs



- ✓Il faut toujours définir un constructeur par défaut.
  - ❑ `public TicketMachine(){…}`
- ✓Il peut y avoir plusieurs constructeurs ou méthodes de même nom si la liste des paramètres (nombre, type) **diffère**.
- ✓Le constructeur le plus générale fait appel au constructeur plus spécifique
- ✓Astuce : déclarez le constructeur le plus spécifique en premier
  - ❑ `public TicketMachine(int price, String name){…}`
  - ❑ `public TicketMachine(int price){this(price, "defaultName");}`
  - ❑ `public TicketMachine(){this(10);}`

* Création d'instances



- ✓ Une instance est référencée par une (des) variable(s) :
  - ❑ `Voiture maFerrari = new Voiture("Ferrari", "rouge");`
  - ❑ `Voiture maTititne = new Voiture();`
  - ❑ `Voiture taFerrari = maFerrari;`

* Méthodes

- ✓ ***Méthode*** : fonction ou procédure nommée, **définie au sein d'une classe** et définissant une propriété fonctionnelle des instances de la classe. Elle décrit comment une instance de cette classe va réagir à la réception du message correspondant à son nom.

- ✓ Les méthodes ont des ***paramètres*** afin de passer des données à appliquer :
  - ❑ *accélérer (int vitesse)* - ***maFerrari.accelerer(30);***
  - ❑ *déplacer(int distance)* - ***leCarre.déplacer(10);***
  - ❑ *placer(Pion unPion, Case uneCase)*

* Méthodes accesseurs (1/2)

- ✓ Les ***accesseurs*** sont un type de méthode spécifique qui fournit les informations (valeurs des variables d'instances) d'un objet;

```
public int getXPosition()
{
 return xPosition;
}
```

```
public String getColor()
{
 return color;
}
```

- ✓ Type de retour : `int, String` …
  - ❑ La méthode renvoie une valeur correspondant au type de retour
- ✓ Nom de méthode : `getXPosition, getColor`



Convention de nommage

* Méthodes accesseurs (2/2)

- ✓ Liste de paramètres : vide
  - ❑ L'information est dans l'instance on n'a pas besoin de données extérieures.
- ✓ Marqueurs du Bloc de début et fin de la méthode : `{ }`
- ✓ Visibilité : `public`
  - ❑ Peut être utilisée à l'extérieur de la classe, dans une autre classe.
- ✓ Instruction : instruction `return`
  - ❑ Il y a toujours un `return` quand une méthode renvoie une valeur de retour;
  - ❑ C'est la dernière instruction exécutée.



Les `accesseurs` sont des méthodes permettant de consulter la valeur de l'attribut de chaque objet de la classe.

* Méthodes mutator (1/2)



✓ Un ***mutator*** est une méthode spécifique pour changer l'état d'un objet …

- ❑ Contient une instruction d'affectation : affectation des variables d'instances;
- ❑ Reçoit des paramètres : valeur à affecter.

✓ Exemple :

```
public void setPrice(int prix){
        price=prix;
    }
```

* Méthodes mutator (2/2)

- ✓ Type de retour (`void`) : ne renvoie rien
- ✓ Nom de méthodes : `setXPosition, setColor(String x)`



Convention de nommage

- ✓ Paramètres : `int x, String x`
- ✓ Visibilité : `public`

- ✓ Instruction
  - ❑ Affectation : `xPosition = x;`



Les `mutators` permettent de modifier les valeurs de l'attribut de chaque objet de la classe.

* Attributs ou variables d'instances (1/2)

- ✓ Les ***attributs (variables d'instances)*** sont les données stockées dans les instances;

- ✓ Les ***attributs*** qui décrivent une instance sont définis dans la classe; chaque instance ayant sa valeur propre. On parle de l'***état*** d'un objet.

  - ❑ La couleur, la marque, la vitesse des voitures : *Ma Ferrari est rouge et roule à 130*;
  - ❑ Les coordonnées et la taille d'un côté d'un carré.

* Attributs ou variables d'instances (2/2)



Une ***variable d'instances*** est définie par sa visibilité, son type, son nom et une éventuelle valeur par défaut;

```
public class TicketMachine{
        private int price;
        private int balance;
        private String color;
        …
}
```

✓ Visibilité : `private, public, protected`
- ❑ Une variable d'instance est toujours privée à priori



✓ Type de données : un type de base, une classe Java Standard ou une classe définie par le programmeur
- ❑ `int, boolean, float, String`, etc.

✓ Nom de variable — *Attention aux conventions de nommage*
- ❑ Commence par une minuscule, séparation des mots par une majuscule



✓ Les valeurs par défaut sont données dans les constructeurs;



✓ Ne pas utiliser à priori l'affectation à la définition.

Variables Locales



- ✓Une variable est locale si elle est déclarée dans la porté
  - ❑d'une méthode
  - ❑d'un bloc
- ✓La variable locale est détruit par le « garbage collector » après l'exécution de la méthode ou bloc

```
public void doSomething(int anyVal){
     int varLocale = 10;
     for (int i=0;i < varLocale; i++){
        // useful things
        // i is also a local variable
     }
}
```

Espace de visibilité de variables

Nom	Déclaré dans	Persistance	Scope
Attribut, Variable d'instance	Classe, hors méthodes	Tps de vie de l'objet	Toute la classe
Paramètre, Argument	Signature de la méthode	Tps de vie méthode	Méthode
Variable locale, Argument	Méthode/bloc	Tps de vie méthode/bloc	Méthode/bloc

* Affichage (1/2)

- ✓ Il existe une méthode prédéfinie que l'on doit redéfinir dans les classes que l'on écrit pour pouvoir afficher les instances d'une classe;
- ✓ Il s'agit de la méthode de nom ***`toString`***;
- ✓ Sa « ***signature*** » doit obligatoirement être écrite de la façon suivante (exemple ci-dessous), car cela permet d'écraser la fonction `toString` déjà existante pour chaque objet mais qui renvoie d'origine l'emplacement en mémoire de l'objet.

```
public String toString(){
        return "balance :" + balance + "\n prix :" + price + "\n";
    }
```

* Affichage (2/2)

- ✓ L'opérateur arithmétique **+** entre deux chaines permet la concaténation des chaînes, entre deux entiers l'addition de ces entiers;

- ✓ L'ajout d'une nouvelle ligne se fait via l'ajout du caractère non-imprimable **\n**;

- ✓ L'instruction correspondant à la méthode prédéfinie **System.out.println**, permet d'écrire une chaîne de caractères dans la console, ajoutant une nouvelle ligne à la fin.

```
/** * Total de la machine à tickets : 20€ */
      System.out.println("Argent gagné :  » + test.emptyMachine());
```

La fonction main (1/2)

```
public static void main(String[] args){…}
```

- ✓ Les arguments de `main` sont des chaînes de caractères, rangées dans le tableau `args`;
- ✓ C'est la fonction appelée à l'exécution d'un programme Java;
- ✓ Si on fait un programme « exécutable », il faut obligatoirement *au moins* une fonction `main`.

La fonction main (2/2)

```
public static void main(String[] args) {
        /** * Le ticket est à 10€ */
       TicketMachine test = new TicketMachine(10);
        /** * Le client donne 5€ ... */
       test.insertMoney(5);
        /** *... mais il n'y a pas assez ... */
       test.getTicket();
        /** * Le client redonne 8€ */
       test.insertMoney(8);
        /** * 5 + 8 = 13 donc on rembourse 3€ */
       test.getTicket();
}
```

Les Données Primitifs Java

Types de données non-objets

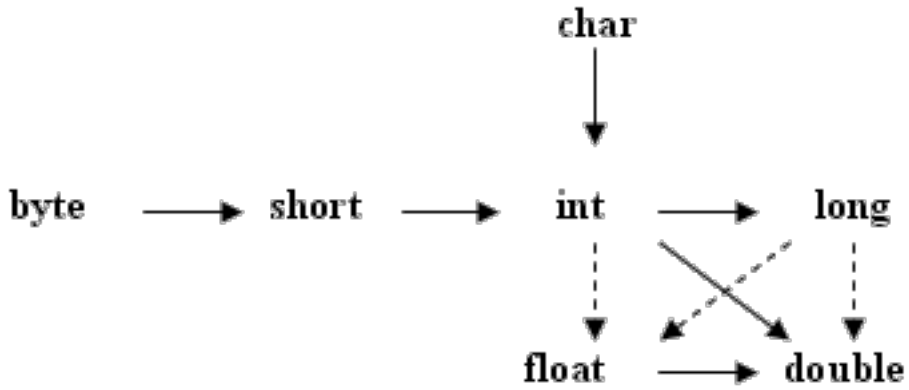


- ✓Les types de données des attributs ou variables peuvent être des objets
  - ❑E.g. String s; TicketMachine tm;
- ✓Tout repose sur de données primitifs -> non-objets
  - ❑À chaque type de donnée primitif correspond un « enveloppe »

Les types de données primitifs

- Booléen : « boolean »
- Caractère : « char »
- Entiers
  - byte (1 octet) : -128 et +127 (-27 et 27-1)
  - short (2 octets) : -32768 et +32767 (-2^{15} et $2^{15}-1$)
  - int (4 octets) : (-2^{31} et $2^{31}-1$)
  - long (8 octets) : (-2^{63} et $2^{63}-1$)
- Flottant
  - float
  - double

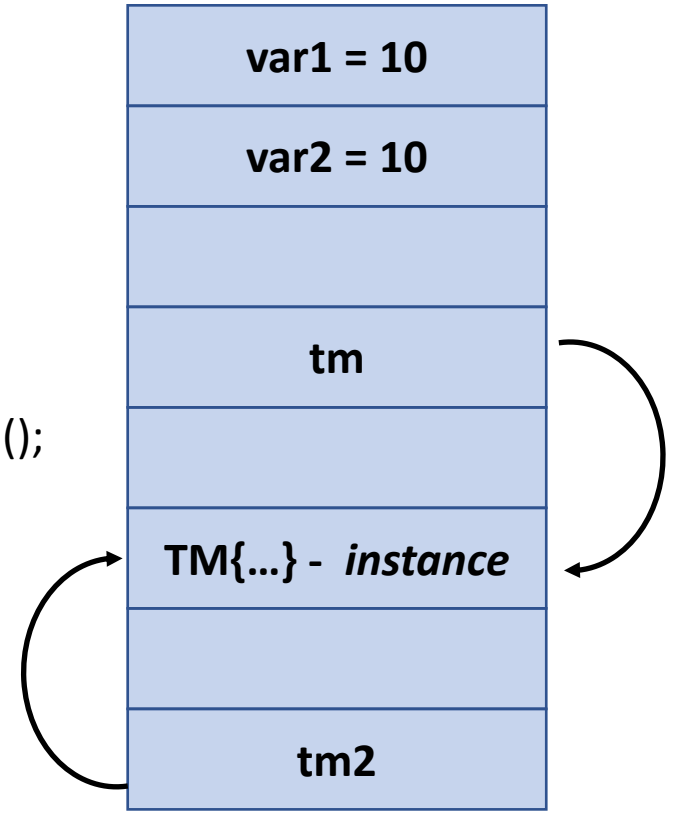
La conversion d'un type primitif à l'autre est possible dans les sens des flèches, sont perte d'information.



Objets vs types primitifs

- Les objets sont échangés par reference
  - On s'échange l'adresse mémoire où "vit" l'instance

```
var1 = 10;
var2 = var1;
tm = new TM();
tm2 = tm;
```



- Les types primitifs s'échangent par valeur
  - On duplique l'information

Questions et remerciements

« *Le pire, quand on fait un discours, ce n'est pas de prendre conscience que vos auditeurs regardent leur montre, c'est le moment où ils se mettent à la secouer pour voir si elle n'est pas arrêtée.* »

Remerciements : Les diapositives où il y a un astérix (*) sont inspirées du précédent cours des collègues suivants. Un grand merci à eux :

Catherine Faron Zucker (*Catherine.FARON-ZUCKER@univ-cotedazur.fr*)

Igor Litovsky (*Igor.LITOVSKY@univ-cotedazur.fr*)

Anne-Marie Pinna Dery (*Anne-Marie.PINNA@univ-cotedazur.fr*)

Frédéric Precioso (*Frederic.PRECIOSO@univ-cotedazur.fr*)

Xavier Blanc (*Xavier.Blanc@labri.fr*)